

IA: MACHINE LEARNING FOUNDATIONS

Support Vector Machines

Adrián Sánchez Morales

SESSION OVERVIEW

LEARNING OBJECTIVES

By the end of this session you will be able to:

1. Explain what a hyperplane is and why SVMs look for the widest possible margin.
2. Describe the role of support vectors and slack variables.
3. Understand how kernels let a linear method draw non-linear boundaries.
4. Tune the key hyperparameters (C and γ) with cross-validation.
5. Extend SVMs to more than two classes and to regression.

AGENDA

1. Maximal Margin Classifier
    - 1.1. Hyperplane
    - 1.2. Maximal Margin Classifier
2. Support Vector Classifier
    - 2.1. Overview
    - 2.2. Details
    - 2.3. Tuning C
3. Support Vector Machines
    - 3.1. Non-Linear Decision Boundaries
    - 3.2. The Kernel Trick
    - 3.3. Kernel Functions
    - 3.4. Pros, Cons & Applications
4. Multiclass SVMs
5. SVMs in Practice
    - 5.1. Practical Tips
    - 5.2. SVM vs Other Models
    - 5.3. Support Vector Regression



1. MAXIMAL MARGIN CLASSIFIER

1.1. HYPERPLANE

- *Support Vector Machine*, an approach mainly for classification, was developed in the computer science community in the 1990s.
- They can be applied in many applications.
- The support vector machine is a generalization of a simple and intuitive classifier called the *maximal margin classifier*.
- In a p-dimensional space, a *hyperplane* is a flat affine subspace of dimension $p - 1$.
- In 2D ⟶ Line
- In 3D ⟶ Plane

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0$$

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p < 0 \quad \Big| \quad \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p > 0.$$

1. MAXIMAL MARGIN CLASSIFIER

1.1. HYPERPLANE

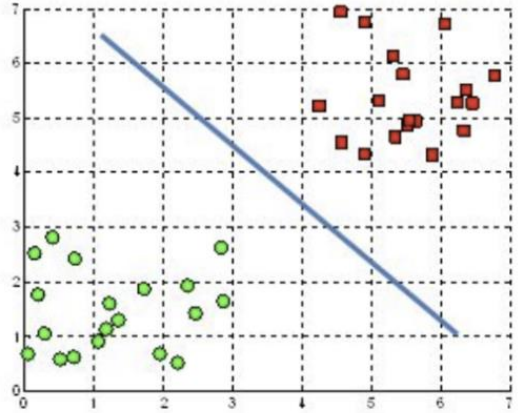
- Classification with a hyperplane is very simple – Perceptron Learning Algorithm

$$x_1 = \begin{pmatrix} x_{11} \\ \vdots \\ x_{1p} \end{pmatrix}, \ldots, x_n = \begin{pmatrix} x_{n1} \\ \vdots \\ x_{np} \end{pmatrix} \qquad y_1, \ldots, y_n \in \{-1, 1\}$$

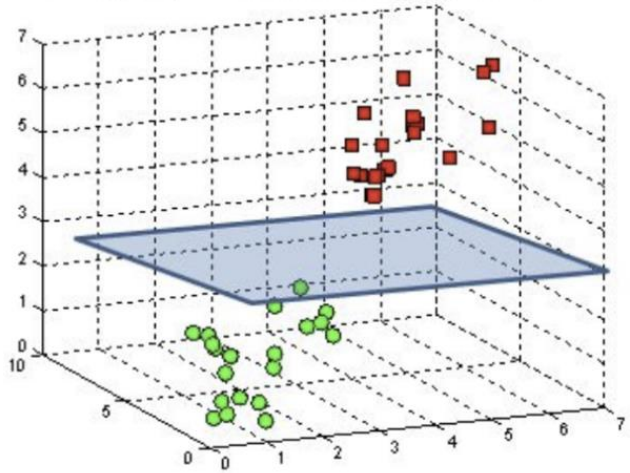
- The idea is to construct a hyperplane that separates the training observations perfectly according to their class labels.

- A new test observation is assigned a class depending on which side of the hyperplane it is located.

A hyperplane in $\mathbb{R}^2$ is a line



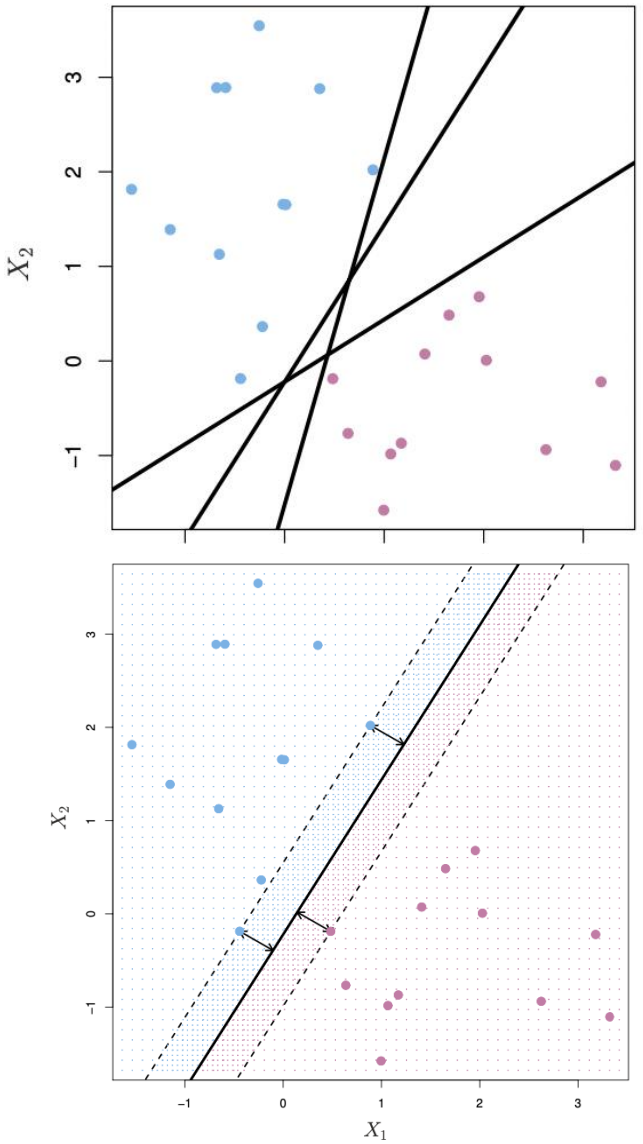
A hyperplane in $\mathbb{R}^3$ is a plane



1. MAXIMAL MARGIN CLASSIFIER

1.2. MAXIMAL MARGIN CLASSIFIER

- The *maximal* margin classifier is the separating hyperplane that is farthest from the training observations.
- It gets the minimal (perpendicular) distance between the observations and the separating hyperplane—that is, a larger margin.
- We can then classify a test observation based on which side of the maximal margin hyperplane it lies.
- We hope that a classifier that has a large margin on the training data will also have a large margin on the test data.



1. MAXIMAL MARGIN CLASSIFIER

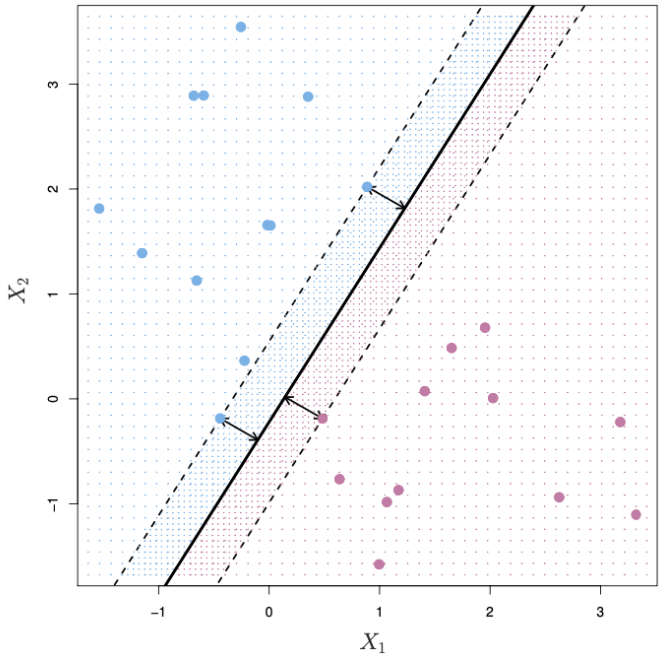
1.2. MAXIMAL MARGIN CLASSIFIER

- There are some observations that are equidistant from the maximal margin hyperplane and define the width of the margin.
- These observations are called *support vectors*. They support the hyperplane.
- The maximal margin hyperplane depends directly on support vectors.
- Constructing the maximal margin classifier is about solving an optimization problem:

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,M}{\text{maximize}} \; M$$

$$\text{subject to} \sum_{j=1}^{p} \beta_j^2 = 1,$$

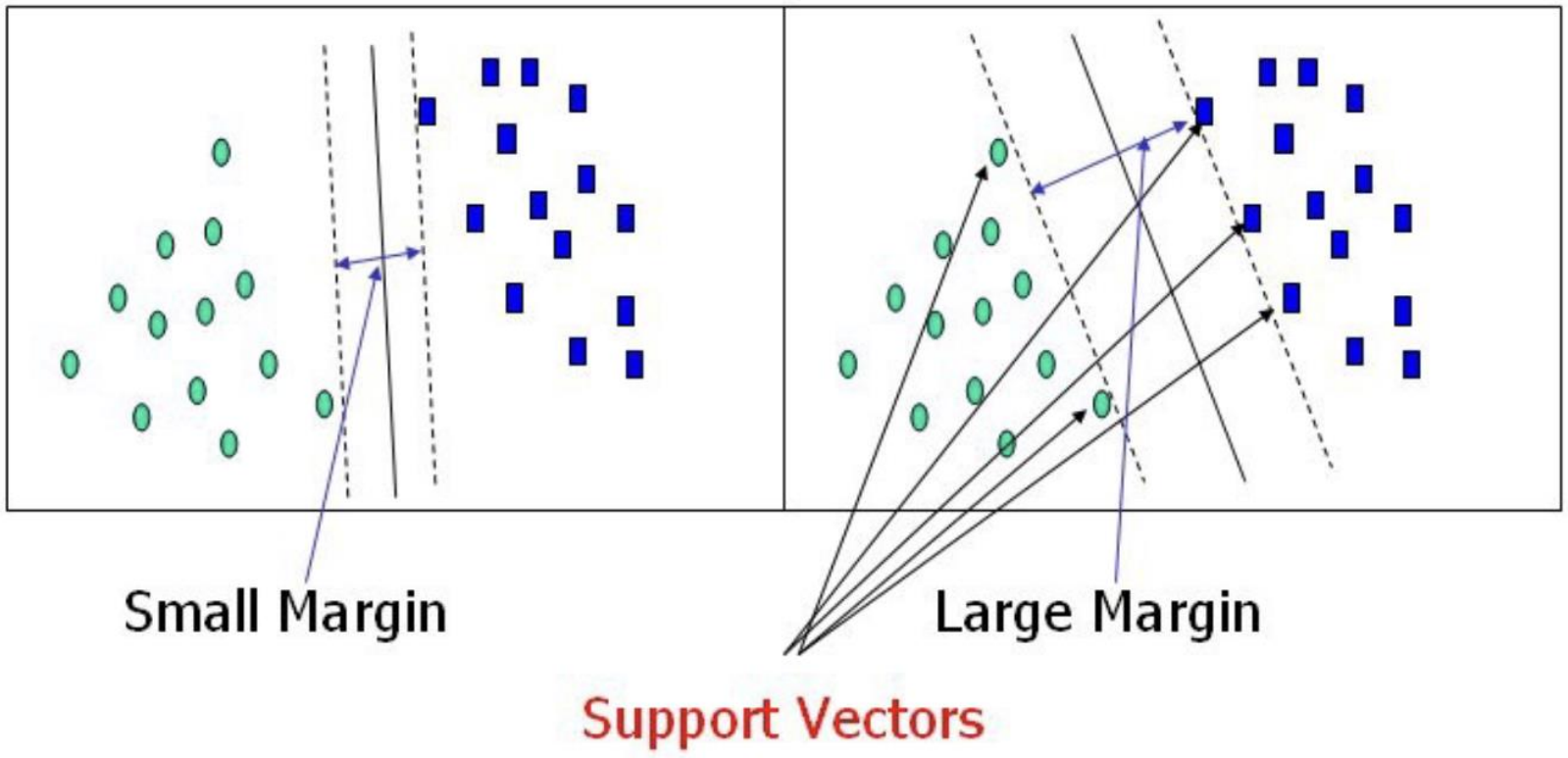
$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M \;\; \forall\, i = 1, \ldots, n$$



M = width of the margin. Every observation must be on the correct side of the hyperplane and at a distance of at least M from it.

1. MAXIMAL MARGIN CLASSIFIER

1.2. MAXIMAL MARGIN CLASSIFIER



1. MAXIMAL MARGIN CLASSIFIER

1.2. MAXIMAL MARGIN CLASSIFIER

DISCUSSION

Difference between PLA and Maximal Margin Classifier?

AGENDA

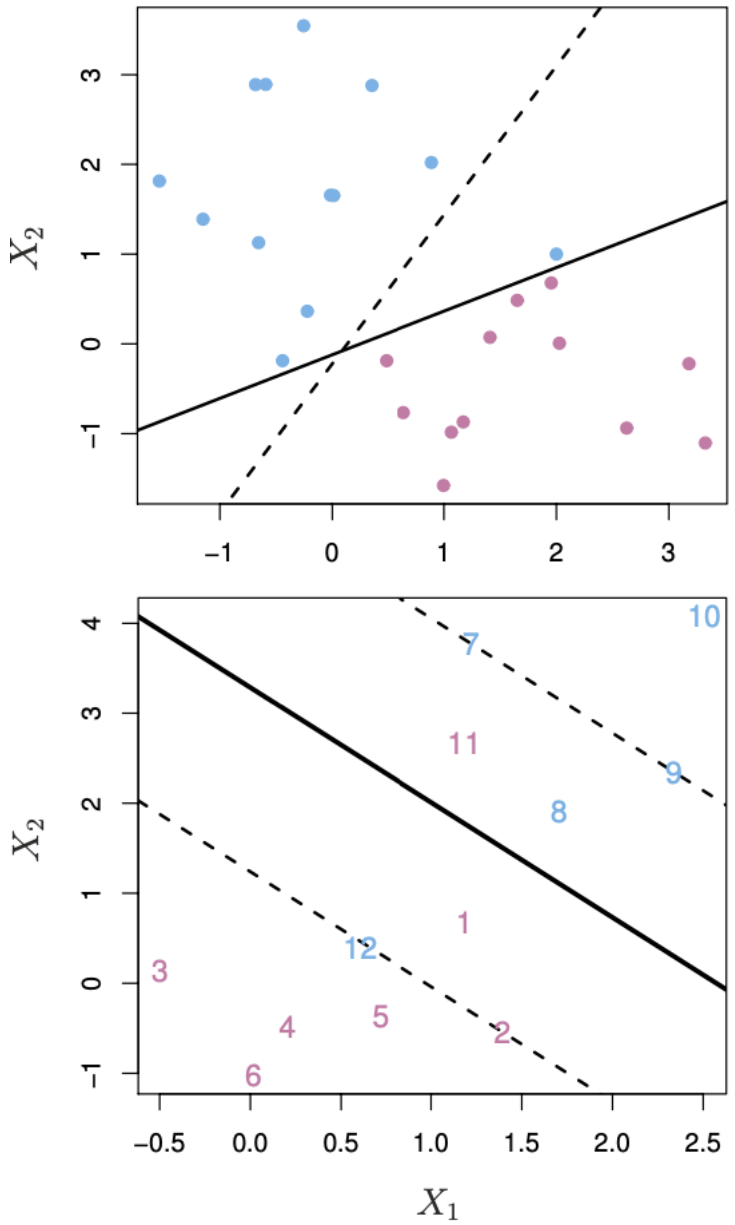
1. Maximal Margin Classifier
   - 1.1. Hyperplane
   - 1.2. Maximal Margin Classifier
2. Support Vector Classifier
   - 2.1. Overview
   - 2.2. Details
   - 2.3. Tuning C
3. Support Vector Machines
   - 3.1. Non-Linear Decision Boundaries
   - 3.2. The Kernel Trick
   - 3.3. Kernel Functions
   - 3.4. Pros, Cons & Applications
4. Multiclass SVMs
5. SVMs in Practice
   - 5.1. Practical Tips
   - 5.2. SVM vs Other Models
   - 5.3. Support Vector Regression



2. SUPPORT VECTOR CLASSIFIER

2.1. OVERVIEW

- What if the classes are not linearly separable?
- What if there exist not desirable observations?
- Do we have overfitting in these cases?
- We may be interested in having a “not perfect” classifier: misclassify a few training observations in order to do a better job in classifying the remaining observations.
- *Support Vector Classifier*



2. SUPPORT VECTOR CLASSIFIER

2.2. DETAILS

- Support Vector Classifier chooses the hyperplane to correctly classify most of the samples but may misclassify a few observations.

$$\underset{\beta_0,\beta_1,\ldots,\beta_p,\epsilon_1,\ldots,\epsilon_n,\,M}{\text{maximize}} \quad M$$

$$\text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}) \geq M(1 - \epsilon_i),$$

$$\epsilon_i \geq 0, \quad \sum_{i=1}^{n} \epsilon_i \leq C,$$

$\epsilon_1, \ldots, \epsilon_n$ *Slack variables*

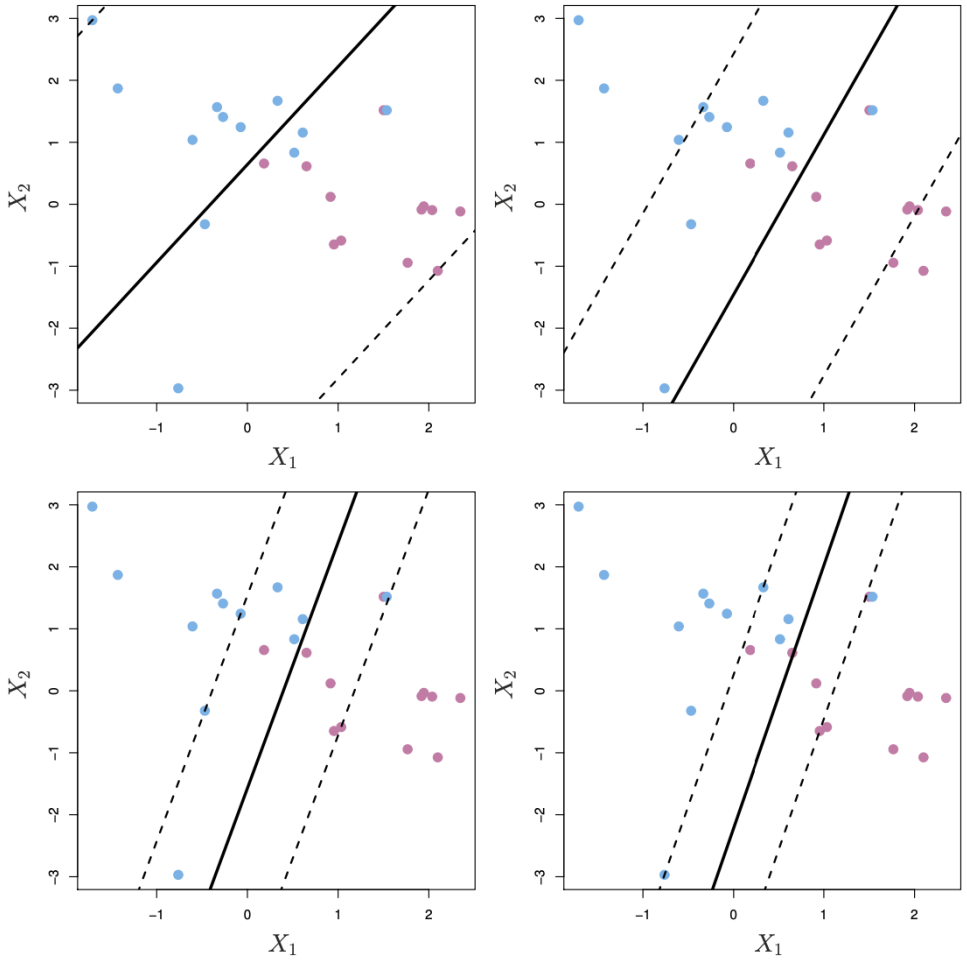
- $\epsilon_i = 0 \longrightarrow$ Correct classification out of the margin.
- $\epsilon_i > 0 \longrightarrow$ Correct classification inside the margin.
- $\epsilon_i > 1 \longrightarrow$ Misclassified

C *Budget* – total amount of margin violation we allow. It acts as a regularization parameter (see 2.3).

2. SUPPORT VECTOR CLASSIFIER

2.2. DETAILS

- Only observations that lie on the margin or violate it affect the hyperplane: these are the ***support vectors***.
- The support vector classifier is quite robust to observations that are far away from the hyperplane. This is different from methods such as LDA, which use all the observations.
- **Figure:** same data, four values of C. The largest C is at the top-left and C decreases in the other panels: the margin narrows and fewer observations become support vectors.



2. SUPPORT VECTOR CLASSIFIER

2.3. TUNING THE PARAMETER C

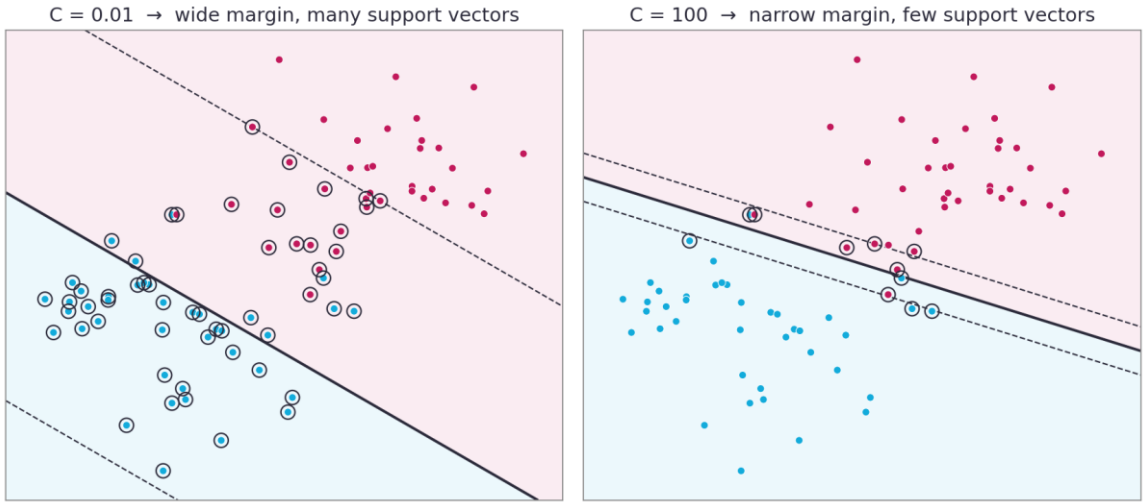
C controls how many margin violations we tolerate: the **bias–variance trade-off** of the classifier.

In some books (budget C, slide 11)

- **Large C** → more violations allowed → wider margin, more support vectors → higher bias, lower variance.
- **Small C** → few violations → narrow margin → lower bias, higher variance.

Careful: scikit-learn uses C the other way round

- C is a **penalty** on violations: **small C** → wide margin (more regularization); **large C** → narrow margin that tries to classify every training point.
- Default: C = 1. Choose it with cross-validation.



Linear SVM fitted with scikit-learn's convention on simulated data. Circled points are support vectors.

AGENDA

1. Maximal Margin Classifier
   - 1.1. Hyperplane
   - 1.2. Maximal Margin Classifier
2. Support Vector Classifier
   - 2.1. Overview
   - 2.2. Details
   - 2.3. Tuning C
3. Support Vector Machines
   - 3.1. Non-Linear Decision Boundaries
   - 3.2. The Kernel Trick
   - 3.3. Kernel Functions
   - 3.4. Pros, Cons & Applications
4. Multiclass SVMs
5. SVMs in Practice
   - 5.1. Practical Tips
   - 5.2. SVM vs Other Models
   - 5.3. Support Vector Regression



3. SUPPORT VECTOR MACHINES

3.1. NON-LINEAR DECISION BOUNDARIES

- In real life, it is not frequent to face linear problems.
- Here, as we did with other algorithms, we may enlarge the feature space to use non-linear boundaries: quadratic, cubic or even higher-order polynomial functions.

$$X_1, X_2, \ldots, X_p \longrightarrow X_1, X_1^2, X_2, X_2^2, \ldots, X_p, X_p^2$$

- Classifying with non-linear decision boundaries:

$$\underset{\beta_0,\beta_{11},\beta_{12},\ldots,\beta_{p1},\beta_{p2},\epsilon_1,\ldots,\epsilon_n,\,M}{\text{maximize}} \; M$$

$$\text{subject to } y_i \left(\beta_0 + \sum_{j=1}^{p} \beta_{j1} x_{ij} + \sum_{j=1}^{p} \beta_{j2} x_{ij}^2 \right) \geq M(1 - \epsilon_i),$$

$$\sum_{i=1}^{n} \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \sum_{j=1}^{p} \sum_{k=1}^{2} \beta_{jk}^2 = 1.$$

There are many ways to enlarge the feature space

It is computationally expensive

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK

- The *support vector machine* (SVM) is an extension of the support vector classifier that results from enlarging the feature space using kernels.

- What is a kernel function?



This mapping is called *kernel function*

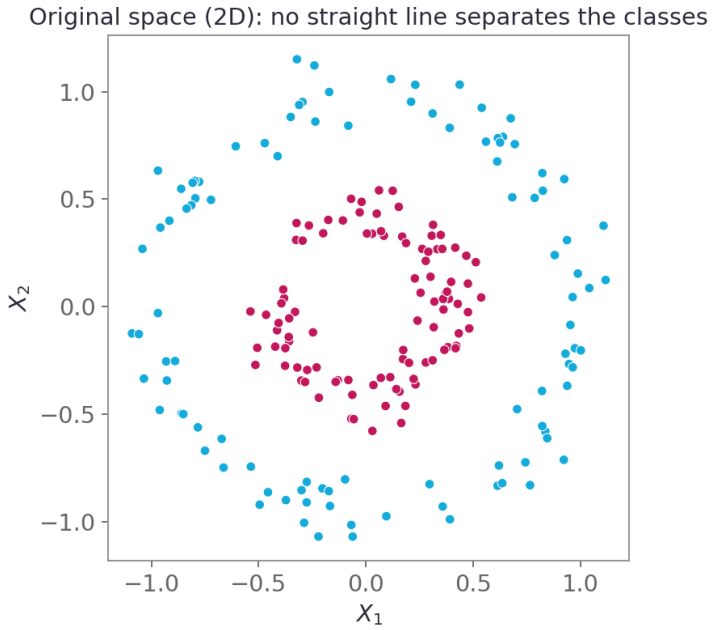
Kernel Trick

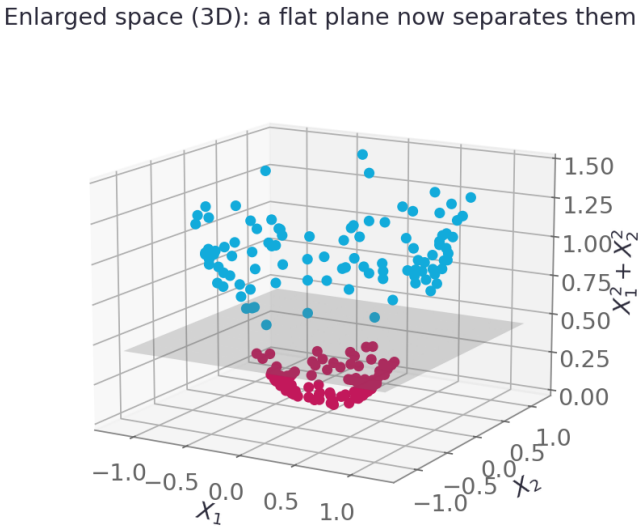
How does the kernel trick work?

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK

- **Idea:** move the data to a higher-dimensional space where the classes can be separated by a flat hyperplane.
- Example: add a third feature $\mathbf{Z = X_1^2 + X_2^2}$. Inner points have a small Z, outer points a large Z.
- A flat plane in 3D becomes a **circle** when we go back to 2D → a non-linear boundary in the original space.
- Problem: with many features, building all these new features explicitly is **very expensive**.



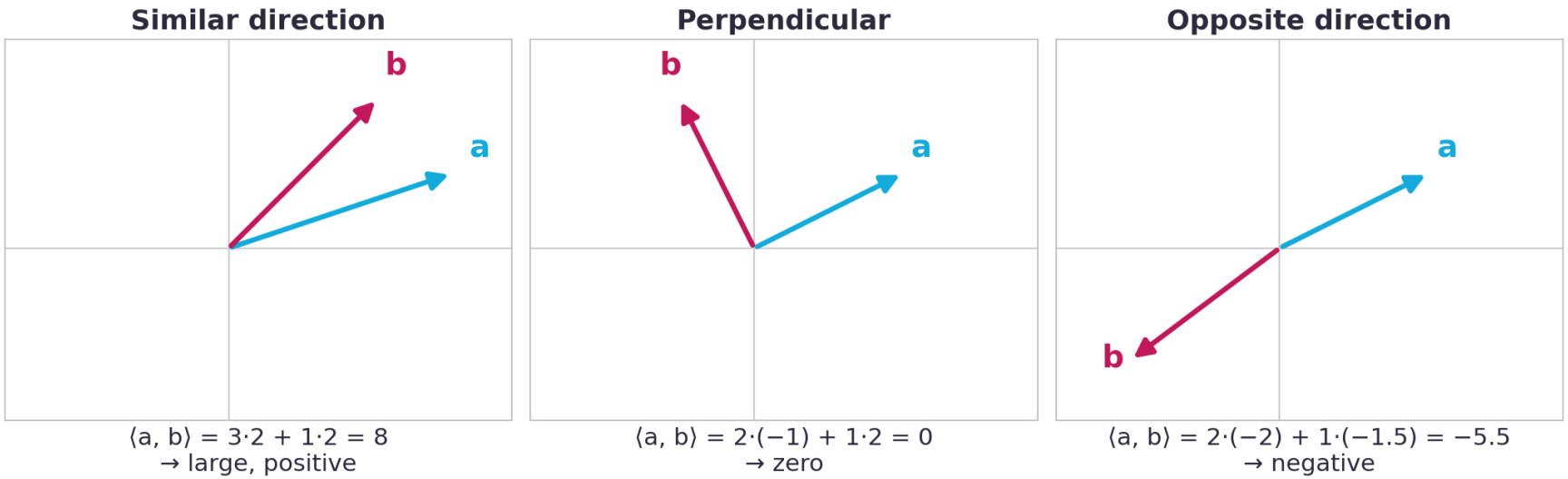


3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK – STEP 1: MEASURING SIMILARITY

To understand the trick we need one ingredient: the **inner product** (or dot product) of two observations. Multiply them feature by feature and add up the results:

$$\langle a, b \rangle = a_1 \cdot b_1 + a_2 \cdot b_2 + \ldots + a_p \cdot b_p$$



Intuition: the inner product works as a **similarity score**. It is large when two observations point in the same direction, zero when they are unrelated (perpendicular) and negative when they point in opposite directions.

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK – STEP 2: THE SVM ONLY NEEDS SIMILARITIES

Key fact (ISL 9.3.2): the support vector classifier can be written using **only inner products** between the new observation x and the support vectors x_i:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \alpha_i \langle x, x_i \rangle$$

β_0

A constant: shifts the hyperplane (the intercept).

α_i

How important support vector i is. Learned during training. $\alpha_i = 0$ for every observation that is not a support vector.

$\langle x, x_i \rangle$

Similarity between the new observation x and support vector x_i (Step 1).

In plain words: the prediction is a **weighted vote of the support vectors**. Each support vector votes for its own class, and its vote counts more when the new observation is similar to it. If f(x) > 0 we predict one class, if f(x) < 0 the other.

Consequence: if we change how we measure similarity, we change the shape of the decision boundary.

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK – STEP 3: THE TRICK, WITH NUMBERS

Suppose we enlarge 2 features with all degree-2 terms: $\varphi(x) = (x_1^2, \sqrt{2}\cdot x_1 x_2, x_2^2)$ → 2 features become 3. Take **a = (1, 2)** and **b = (3, 1)**. How similar are they in the enlarged space?

Route 1 – the long way

Build the new features, then compare

1. $\varphi(a) = (1^2, \sqrt{2}\cdot 1\cdot 2, 2^2) = (1, 2\sqrt{2}, 4)$
2. $\varphi(b) = (3^2, \sqrt{2}\cdot 3\cdot 1, 1^2) = (9, 3\sqrt{2}, 1)$
3. $\langle \varphi(a), \varphi(b) \rangle = 1\cdot 9 + 2\sqrt{2}\cdot 3\sqrt{2} + 4\cdot 1$
   $= 9 + 12 + 4 =$ **25**

=

Route 2 – the kernel trick

Stay in the original 2 features

1. $\langle a, b \rangle = 1\cdot 3 + 2\cdot 1 = 5$
2. $K(a, b) = \langle a, b \rangle^2 = 5^2 =$ **25**
   No new features were built.

Same answer, much less work. The kernel $K(a, b) = \langle a, b \rangle^2$ gives the similarity in the enlarged space while only computing in the original space. This is the polynomial kernel with d = 2 (γ = 1, r = 0). The SVM simply uses K instead of ⟨ , ⟩ in the formula of Step 2.

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK – STEP 4: WHY IT MATTERS

With 2 features the saving looks small. With real data it is huge:

500,500

new features if we add all pairwise products of 1,000 original features (Route 1).

≈ 1,000

multiplications with the kernel: one inner product in the original space, then square it (Route 2).

∞

features implicitly used by the RBF kernel. Impossible to build, yet the kernel itself is a single, cheap formula (see 3.3).

Putting it all together: how an SVM with a kernel classifies a new observation

1. New observation x

Only its original features. No new features are created.

→

2. Compute $K(x, x_i)$

Similarity of x to each support vector, measured with the kernel.

→

3. Weighted vote

$f(x) = \beta_0 + \Sigma\, \alpha_i\, K(x, x_i)$. The sign of f(x) gives the class.

3. SUPPORT VECTOR MACHINES

3.2. THE KERNEL TRICK – STEP 5: HOW IS THE SVM TRAINED?

1 Compute the kernel matrix
$K(x_i, x_j)$ for every pair of training points: an n × n table of similarities. This is all the algorithm sees of the data.

2 Solve the optimisation problem
Find one weight α_i per training point (dual problem, right).

3 Read the support vectors
They are the points with $\alpha_i > 0$. Nobody chooses them: they come out of the optimisation.

4 Compute β_0
Use a support vector lying exactly on the margin, where $y_i \cdot f(x_i) = 1$.

5 Predict
$f(x) = \beta_0 + \Sigma\, \alpha_i\, y_i\, K(x, x_i)$. Only the support vectors contribute.

The dual problem (step 2)

$$\max_{\alpha} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i\, \alpha_j\, y_i\, y_j\, K(x_i, x_j)$$

$$\text{subject to} \quad 0 \le \alpha_i \le C, \qquad \sum_{i=1}^{n} \alpha_i\, y_i = 0$$

Key point: the data only appear inside $K(x_i, x_j)$, so training never builds the enlarged space either. Price to pay: an n × n matrix → slow for large n.

C follows scikit-learn's convention (penalty). In the ISL notation of Step 2, α_i already includes y_i.

3. SUPPORT VECTOR MACHINES

3.3. KERNEL FUNCTIONS

Linear

$$K(x, x') = \langle x,\, x' \rangle$$

Hyperparameters: none (only C).
When: Many features (e.g. text), and always as a first baseline.

Polynomial

$$K(x, x') = (\gamma \langle x,\, x' \rangle + r)^d$$

Hyperparameters: degree d, γ, r (coef0).
When: Boundaries with interactions between features. d = 1 gives back the linear kernel.

Radial basis function (RBF) / Gaussian

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Hyperparameter: γ > 0.
When: Default in scikit-learn. Very flexible and local: only nearby training points matter.

Sigmoid

$$K(x, x') = \tanh(\gamma \langle x,\, x' \rangle + r)$$

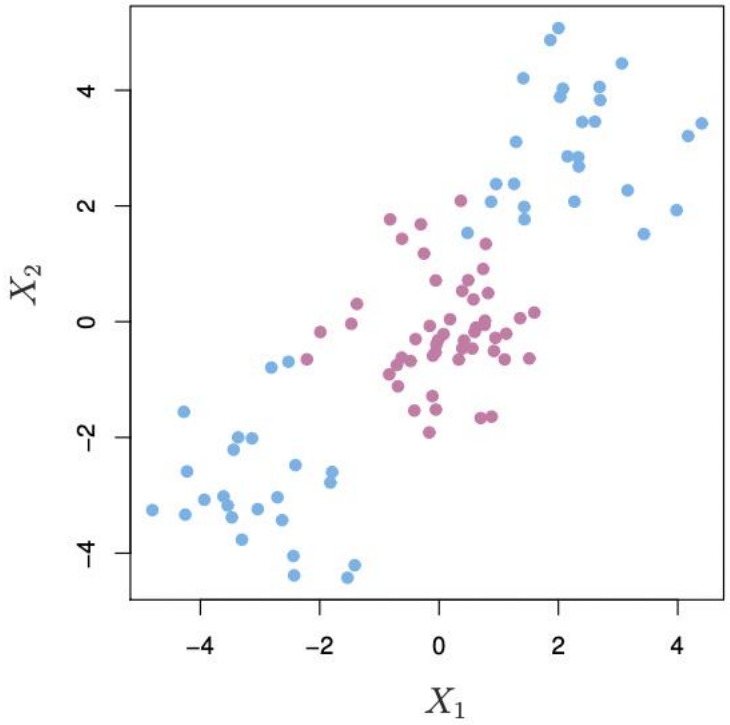
Hyperparameters: γ, r (coef0).
When: Less common. Uses tanh, the same function as a neural network activation.

Note: “Gaussian kernel” and “RBF kernel” are the same thing. Names follow scikit-learn: kernel, degree, gamma, coef0.

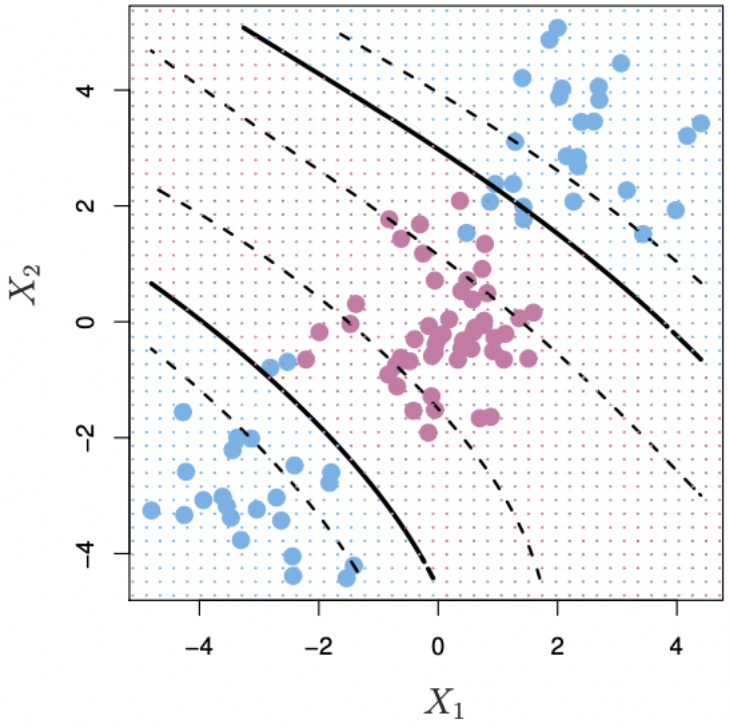
3. SUPPORT VECTOR MACHINES

3.3. KERNEL FUNCTIONS

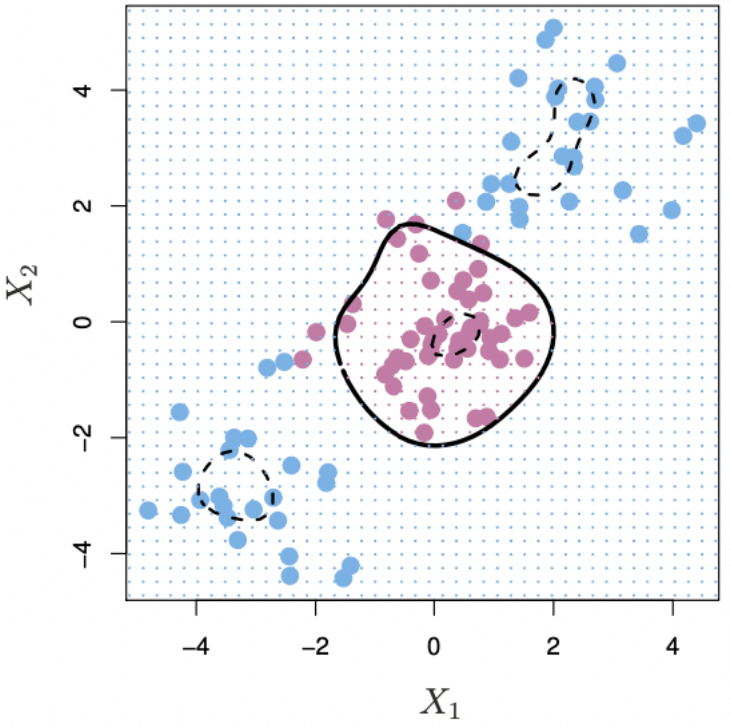
Non-linear Data



Polynomial Kernel



Radial Kernel



3. SUPPORT VECTOR MACHINES

3.4. PROS, CONS & APPLICATIONS

Applications

- Text classification (e.g. spam, topic detection)
- Image classification and face detection
- Handwriting recognition
- Bioinformatics (e.g. protein and gene classification)

Advantages

▲ Effective in high-dimensional spaces, even when there are more features than observations.

▲ Memory-efficient: the model only keeps the support vectors.

▲ Flexible: different kernels for different decision boundaries.

▲ Robust to observations far away from the boundary.

Disadvantages

▼ Kernel SVMs train slowly on large datasets (cost grows between n^2 and n^3).

▼ No direct probability estimates (only approximated, e.g. Platt scaling).

▼ Sensitive to feature scaling and to the choice of kernel, C and γ.

▼ Hard to interpret with non-linear kernels.

AGENDA

1. Maximal Margin Classifier
    - 1.1. Hyperplane
    - 1.2. Maximal Margin Classifier
2. Support Vector Classifier
    - 2.1. Overview
    - 2.2. Details
    - 2.3. Tuning C
3. Support Vector Machines
    - 3.1. Non-Linear Decision Boundaries
    - 3.2. The Kernel Trick
    - 3.3. Kernel Functions
    - 3.4. Pros, Cons & Applications
4. Multiclass SVMs
5. SVMs in Practice
    - 5.1. Practical Tips
    - 5.2. SVM vs Other Models
    - 5.3. Support Vector Regression



4. MULTICLASS SUPPORT VECTOR MACHINES

- It turns out that separating observations by a hyperplane does not make sense in the case of more than two classes.

- Many proposals for extending SVMs to the *K*-class case have been made.

ONE vs ONE Classification

This approach constructs $\binom{K}{2}$ SVM classifiers, each of which compares a pair of classes (binary).

To classify a test sample all the classifiers are used, and it is counted the number of times that the test observation is assigned to each of the K classes.

ONE vs ALL Classification

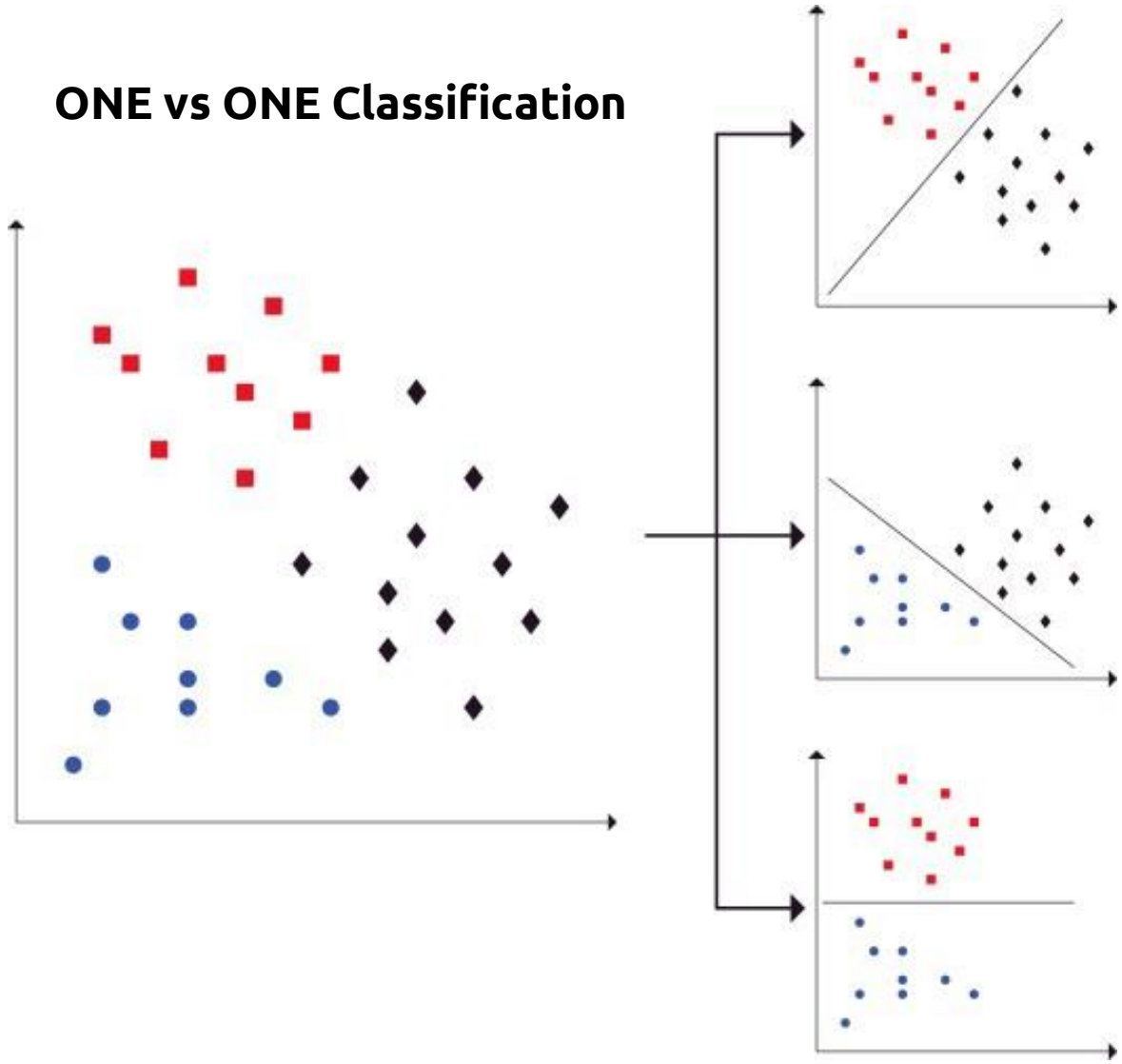
It fits *K* SVMs, each time comparing one of the K classes to the remaining *K* – 1 classes

To classify a test sample:

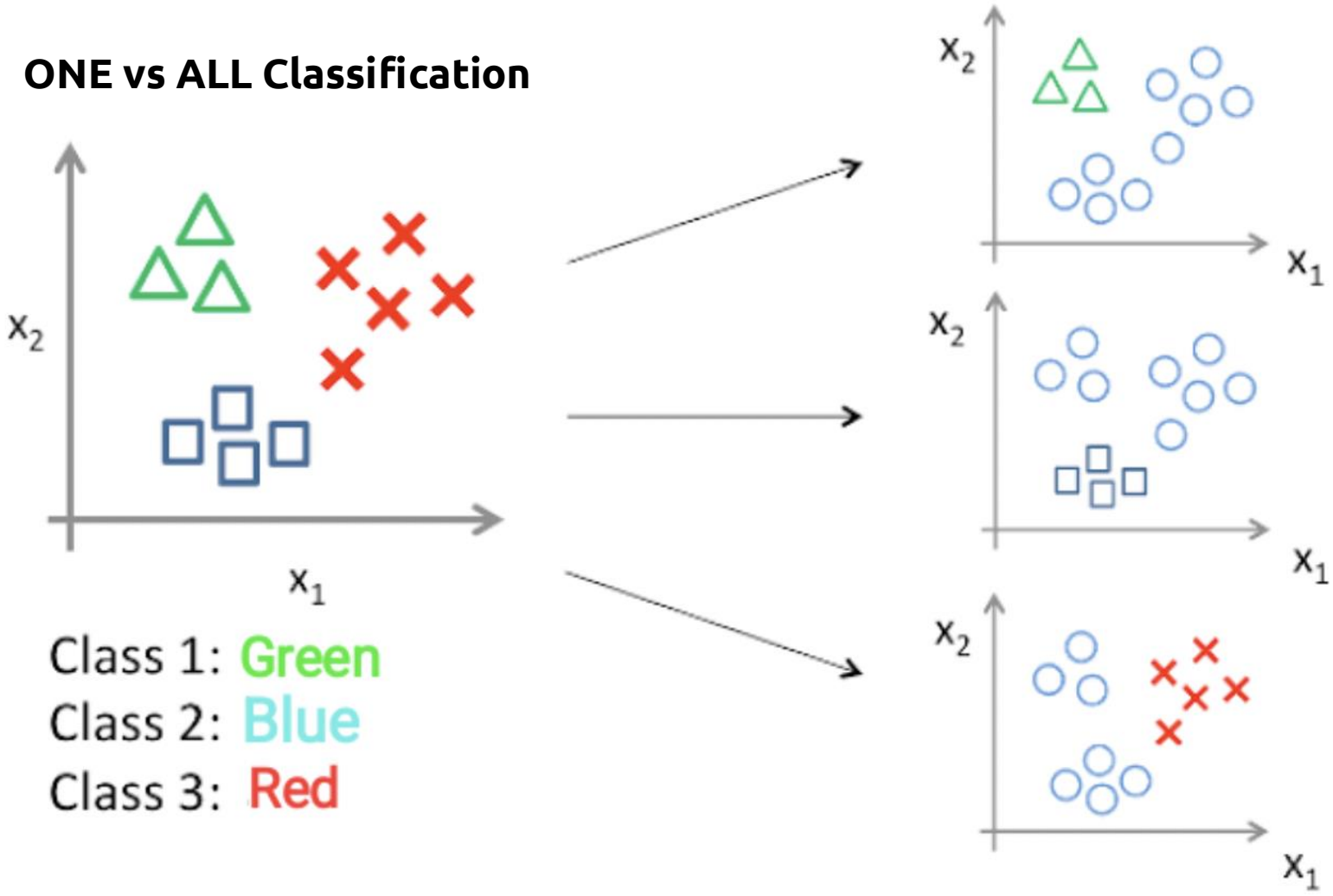
- Let define $\beta_{0k}, \beta_{1k}, \ldots, \beta_{pk}$ as the parameters that result from fitting an SVM comparing the k_{th} class to the others
- We assign the observation to the class with the largest value

$$\beta_{0k} + \beta_{1k}x_1^* + \beta_{2k}x_2^* + \cdots + \beta_{pk}x_p^*$$

4. MULTICLASS SUPPORT VECTOR MACHINES



4. MULTICLASS SUPPORT VECTOR MACHINES



4. MULTICLASS SUPPORT VECTOR MACHINES

QUESTION

Is there any other popular approach?

4. MULTICLASS SUPPORT VECTOR MACHINES

QUESTION

Is there any other popular approach?

Directed Acyclic Graph SVM (DAGSVM)

AGENDA

1. Maximal Margin Classifier
    - 1.1. Hyperplane
    - 1.2. Maximal Margin Classifier
2. Support Vector Classifier
    - 2.1. Overview
    - 2.2. Details
    - 2.3. Tuning C
3. Support Vector Machines
    - 3.1. Non-Linear Decision Boundaries
    - 3.2. The Kernel Trick
    - 3.3. Kernel Functions
    - 3.4. Pros, Cons & Applications
4. Multiclass SVMs
5. SVMs in Practice
    - 5.1. Practical Tips
    - 5.2. SVM vs Other Models
    - 5.3. Support Vector Regression



5. SVMS IN PRACTICE

5.1. PRACTICAL TIPS

1 Scale your features

SVMs are not scale invariant. Standardise (mean 0, variance 1) inside a Pipeline, so the same scaling is applied to the test data.

2 Start simple

Try a linear kernel first (fast, good with many features). Move to RBF if the boundary looks non-linear.

3 Tune C and γ together

GridSearchCV with values on a log scale (0.01 … 100) and k-fold cross-validation (Session 6).

4 Imbalanced classes

class_weight='balanced' gives more weight to errors on the minority class.

5 Large datasets

Kernel SVC training grows at least quadratically with n. Use LinearSVC or SGDClassifier for big data.

6 Need probabilities?

SVC(probability=True) uses Platt scaling (extra internal CV, slower). Otherwise use decision_function scores.

5. SVMS IN PRACTICE

5.1. PRACTICAL TIPS – A COMPLETE WORKFLOW IN SCIKIT-LEARN

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=300, noise=0.3, random_state=0)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=0)

pipe = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = {"svc__C": [0.01, 0.1, 1, 10, 100],
        "svc__gamma": [0.01, 0.1, 1, 10, 100]}
search = GridSearchCV(pipe, grid, cv=5)
search.fit(X_train, y_train)

print(search.best_params_)
print("Test accuracy:", search.score(X_test, y_test))
```

1. **Data**
   Simulated two-class data, split into train and test.

2. **Scale + model**
   The Pipeline scales inside each CV fold, so there is no leakage.

3. **Tune**
   5-fold cross-validation over 25 combinations of C and γ.

4. **Evaluate**
   Report the final score only on the untouched test set.

5. SVMS IN PRACTICE

5.2. SVM VS OTHER MODELS SEEN IN THE COURSE

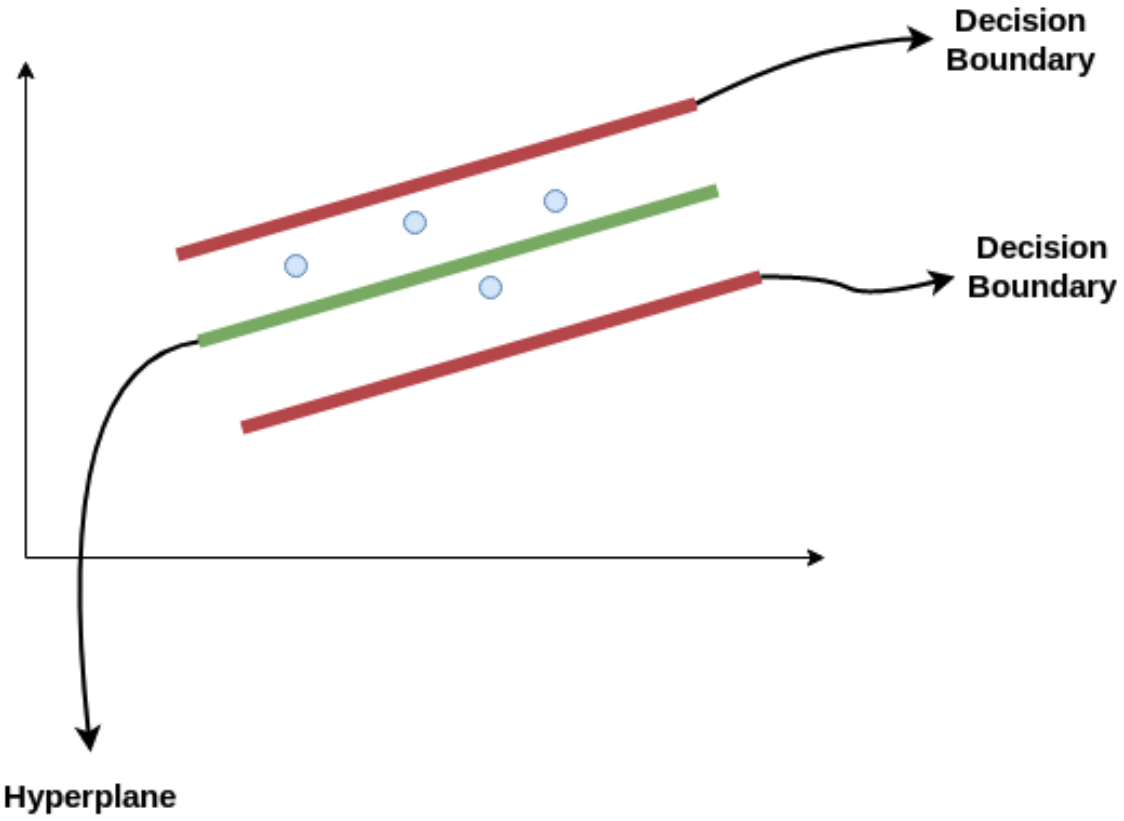
	Logistic regression (S5)	KNN (S5)	Trees / Random forest (S8)	SVM (today)
Decision boundary	Linear (unless we add features)	Any shape (local)	Any shape (axis-parallel splits)	Linear, or any shape with kernels
Probabilities	Yes, directly	Approximate (share of neighbours)	Yes (share of each class in a leaf)	Not directly (Platt scaling)
Interpretability	High (coefficients)	Low	Tree: high / Forest: low	Low with non-linear kernels
Large datasets	Scales well	Slow predictions (stores all data)	Scales well	Kernel SVM: slow to train
Feature scaling	Recommended with regularization	Required (distances)	Not needed	Required

ISL (9.5): *"When the classes are well separated, SVMs tend to behave better than logistic regression; in more overlapping regimes, logistic regression is often preferred."*

5. SVMS IN PRACTICE

5.3. SUPPORT VECTOR REGRESSION

- SVMs can also predict **numbers** (regression).
- Fit a function that keeps as many points as possible inside a **tube of width ±ε** around it.
- Errors smaller than ε are ignored (ε-insensitive loss). Only points on or outside the tube become support vectors.
- Hyperparameters: ε (tube width), C (penalty for points outside the tube) and the kernel (+ γ).
- **scikit-learn:** `SVR(kernel="rbf", C=1.0, epsilon=0.1)`



KEY TAKEAWAYS

SUMMARY

1. SVMs separate classes with the hyperplane that has the **widest margin**. Only the **support vectors** define it.
2. Real data overlap: **slack variables** let some points violate the margin; **C** controls how many (bias–variance).
3. **Kernels** (polynomial, RBF…) give non-linear boundaries at almost the cost of a linear one: the kernel trick.
4. An SVM is **hinge loss + ridge-type penalty**: the same "loss + penalty" recipe as other models in the course.
5. More than two classes: **one-vs-one** or **one-vs-all**. **SVR** extends the idea to regression.
6. In practice: **scale** the features, start **linear**, tune **C and γ** with cross-validation.

REFERENCES

FURTHER READING

- James, G., Witten, D., Hastie, T., Tibshirani, R. & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer. Chapter 9: Support Vector Machines. Free at statlearning.com
- scikit-learn User Guide – 1.4 Support Vector Machines: scikit-learn.org/stable/modules/svm.html
- scikit-learn example – RBF SVM parameters: scikit-learn.org/stable/auto_examples/svm/plot_rbf_parameters.html
- Cortes, C. & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.
- Platt, J., Cristianini, N. & Shawe-Taylor, J. (2000). Large margin DAGs for multiclass classification. *Advances in Neural Information Processing Systems 12*.
- Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*.


THANK YOU